REVISION CHEAT SHEET:

ARRAYS / TUPLES

Arrays, Tuples, and Basic Data Containers in C#

Interview-ready notes · examples · C# · diagrams

Topic 10 of 40

Arrays and lightweight data containers are fundamental because almost every real application needs to store, access, group, transform, and pass multiple values.

In the Fundraising Platform, you might need to represent:

EXAMPLE / FLOW

```
A group of gift amounts

A batch of Gift IDs

Monthly fundraising totals

Campaign statistics

A method result containing Count + Total

A key/value pair such as FundId → Amount

A temporary projection containing
GiftId + DonorName + Amount
```

C# provides several different tools for these situations.

This topic covers:

- arrays;
- array declaration and creation;
- fixed-size behavior;
- default element values;
- array initialization;
- modern collection expressions;
- spread elements;
- indexing;
- zero-based indexes;
- `Length`;
- `Rank`;
- `GetLength`;
- one-dimensional arrays;
- multidimensional arrays;
- rectangular arrays;
- jagged arrays;
- nested arrays;
- iteration using `for`;
- iteration using `foreach`;

- modifying elements;
- reference-type behavior;
- copying arrays;
- shallow copying;
- `Array.Copy`;
- `Clone`;
- `Array.Resize`;
- common `Array` methods;
- sorting;
- reversing;
- searching;
- clearing;
- indexes from the end using `^`;
- ranges using `..`;
- array slicing;
- performance characteristics;
- tuples;
- named tuples;
- tuple return values;
- tuple assignment;
- tuple deconstruction;
- discards;
- tuple equality;
- tuple limitations;
- `KeyValuePair<TKey,TValue>`;
- anonymous types;
- anonymous projections;
- when to use arrays vs tuples vs objects;
- interview coding problems;
- Fundraising Platform examples.

The high-level picture is:

EXAMPLE / FLOW

```
Need Multiple Values
        →
What kind of relationship?
        →
Same Type + Fixed Size
        →
Array

Few Different Values
that belong together
        →
Tuple

Key + Value Pair
        →
KeyValuePair<TKey,TValue>

Temporary Named Projection
        →
Anonymous Type
```

```
Important Business Concept
        →
Named Class / Record / Struct
```

ARRAYS / TUPLES

What Is an Array?

Definition

An **array is a fixed-size sequence of elements of the same element type**.

Example:

C#

```
decimal[] giftAmounts = {
    100m,
    250m,
    500m
};
```

The array contains:

EXAMPLE / FLOW

```
100
250
500
```

Every element is:

EXAMPLE / FLOW

```
decimal
```

An array cannot directly contain a mixture such as:

EXAMPLE / FLOW

```
decimal
string
DateTime
Gift
```

unless you deliberately use a common type such as `object`, which is usually not the best design.

C# arrays are reference types, derive from `System.Array`, are zero-indexed, and have dimensions whose sizes are fixed when the array instance is created.

Basic Array Mental Model

EXAMPLE / FLOW

```
decimal[] giftAmounts

Index      Value

0          100
1          250
2          500
```

The variable:

EXAMPLE / FLOW

```
giftAmounts
```

contains a reference to an array object.

Conceptually:

EXAMPLE / FLOW

```
giftAmounts
    →
Array Object
    →
[100][250][500]
```

Arrays Are Reference Types

This is extremely important.

Consider:

C#

```
int[] first = {
    10,
    20,
    30
};

int[] second = first;
```

This does **not** create another independent array.

Instead:

EXAMPLE / FLOW

```
first  ──┐
         ↓
      [10,20,30]
         ↑
second ──┘
```

Now:

C#

```
second[0] = 999;
```

Then:

C#

```
Console.WriteLine(
    first[0]);
```

prints:

EXAMPLE / FLOW

```
999
```

because both variables refer to the same array.

Array vs Array Element

Do not confuse:

EXAMPLE / FLOW
```
Array type
```

with:

EXAMPLE / FLOW
```
Element type
```

Example:

C#
```
decimal[] values;
```

The array itself:

EXAMPLE / FLOW
```
decimal[]
```

is a reference type.

Each element:

EXAMPLE / FLOW
```
decimal
```

is a value type.

Conceptually:

EXAMPLE / FLOW
```
Reference Variable
    →
Array Object
    →
decimal
decimal
decimal
```

Declaring an Array

Syntax:

C#
```
Type[] variableName;
```

Example:

C#
```
int[] giftIds;
```

C#
```
decimal[] giftAmounts;
```

```C#
string[] campaignNames;
```

```C#
Gift[] gifts;
```

At this point you have declared an array variable.

You have not necessarily created an array instance yet.

Declaring vs Creating

Declaration:

```C#
int[] giftIds;
```

Creation:

```C#
giftIds = new int[5];
```

Combined:

```C#
int[] giftIds = new int[5];
```

Flow:

```EXAMPLE / FLOW
Declare Array Variable
    →
Create Array Object
Length = 5
    →
Assign Reference
```

Fixed Size

When:

```C#
int[] giftIds = new int[5];
```

is created, its length is:

```EXAMPLE / FLOW
5
```

That array instance cannot later become length:

```EXAMPLE / FLOW
10
```

or:

EXAMPLE / FLOW

```
3
```

The rank and dimensions of an existing array instance remain fixed for its lifetime.

This is one of the biggest differences between:

EXAMPLE / FLOW

```
Array
```

and:

EXAMPLE / FLOW

```
List<T>
```

which we will study later.

Array Default Values

Suppose:

C#

```
int[] values = new int[3];
```

You did not explicitly initialize its elements.

The result is:

EXAMPLE / FLOW

```
[0, 0, 0]
```

because:

EXAMPLE / FLOW

```
default(int)
=
0
```

Array elements are initialized to their type's default value when the array is created.

Reference-Type Array Defaults

C#

```
string?[] names = new string?[3];
```

Initially:

EXAMPLE / FLOW

```
[null, null, null]
```

Likewise:

C#

```
Gift?[] gifts = new Gift?[3];
```

initially contains:

EXAMPLE / FLOW

```
[null, null, null]
```


Boolean Array Defaults

C#

```
bool[] flags = new bool[3];
```

Result:

EXAMPLE / FLOW

```
false
false
false
```

Decimal Array Defaults

C#

```
decimal[] amounts = new decimal[3];
```

Result:

EXAMPLE / FLOW

```
0
0
0
```

Array Initialization

Classic syntax:

C#

```
int[] numbers = {
    10,
    20,
    30
};
```

You can also write:

C#

```
int[] numbers = new int[]
    {
        10,
        20,
        30
    };
```

Or:

C#

```
int[] numbers = new[]
    {
        10,
        20,
```

```
        30
    };
```


Here the compiler infers:

```
EXAMPLE / FLOW
int[]
```

Modern Collection Expressions

Modern C# supports collection expressions:

```
C#
int[] numbers = [
    10,
    20,
    30
];
```

Collection expressions were introduced in C# 12 and can initialize arrays and many other collection-like types.

For your modern C# roadmap, this syntax is perfectly valid.

Collection Expression

```
C#
decimal[] giftAmounts = [
    100m,
    250m,
    500m
];
```

This creates:

```
EXAMPLE / FLOW
decimal[]
```

because the target type provides the context.

Spread Element

Modern collection expressions support:

```
EXAMPLE / FLOW
..
```

as a spread element.

Example:

```
C#
int[] first = [
    1,
    2,
    3
];

int[] second = [
```

```
    4,
    5
];

int[] combined = [
    .. first,
    .. second
];
```

Result:

```
EXAMPLE / FLOW
[1,2,3,4,5]
```

Spread elements inline the elements from an enumerable source into the new collection.

Fundraising Spread Example

```
C#
long[] onlineGiftIds = [
    1001,
    1002
];

long[] checkGiftIds = [
    2001,
    2002
];

long[] allGiftIds = [
    .. onlineGiftIds,
    .. checkGiftIds
];
```

Result:

```
EXAMPLE / FLOW
1001
1002
2001
2002
```

Accessing Array Elements

Arrays are:

```
EXAMPLE / FLOW
zero-indexed
```

Example:

```
C#
string[] funds = [
    "General",
    "Research",
    "Education"
];
```

Indexes:

EXAMPLE / FLOW

```
Index    Value

0        General
1        Research
2        Education
```

Access:

C#

```
string firstFund = funds[0];
```

Result:

EXAMPLE / FLOW

```
General
```

Why Zero-Based?

The index represents how many elements come before the selected element.

Therefore:

EXAMPLE / FLOW

```
First element
    →
0 elements before it
    →
index 0
```

Second:

EXAMPLE / FLOW

```
1 element before it
    →
index 1
```

This is the standard indexing model used across C# arrays and many collections.

Last Valid Index

For:

C#

```
int[] values = [
    10,
    20,
    30
];
```

Length:

EXAMPLE / FLOW

```
3
```

Valid indexes:

EXAMPLE / FLOW

```
0
1
2
```

Therefore:

EXAMPLE / FLOW

```
Last Index
=
Length - 1
```

IndexOutOfRangeException

This is invalid:

C#

```
Console.WriteLine(
    values[3]);
```

because:

EXAMPLE / FLOW

```
Length = 3
```

but valid indexes stop at:

EXAMPLE / FLOW

```
2
```

This causes:

EXAMPLE / FLOW

```
IndexOutOfRangeException
```

Modifying an Array Element

Arrays are fixed-size, but their elements can normally be modified.

Example:

C#

```
decimal[] amounts = [
    100m,
    200m,
    300m
];

amounts[1] = 999m;
```

Now:

EXAMPLE / FLOW

```
100
999
300
```

Fixed-size does **not** mean immutable.

It means:

EXAMPLE / FLOW
```
number of positions
```

cannot change for that array instance.

Length Property

C#
```
int[] numbers = [
    10,
    20,
    30
];

int length = numbers.Length;
```

Result:

EXAMPLE / FLOW
```
3
```

`Length` gives the total number of elements across all dimensions.

Iterating with for

C#
```
decimal[] giftAmounts = [
    100m,
    250m,
    500m
];

for (int i = 0;
     i < giftAmounts.Length;
     i++)
{
    Console.WriteLine(
        giftAmounts[i]);
}
```

Flow:

EXAMPLE / FLOW
```
i = 0
    →
giftAmounts[0]

i = 1
    →
giftAmounts[1]

i = 2
    →
giftAmounts[2]

i = 3
```

```
    →
condition false
    →
stop
```

Why < Length?

Correct:

```C#
i < giftAmounts.Length
```

Wrong:

```C#
i <= giftAmounts.Length
```

If:

```EXAMPLE / FLOW
Length = 3
```

then `<=` eventually attempts:

```EXAMPLE / FLOW
index 3
```

which is invalid.

Iterating with foreach

```C#
foreach (decimal amount
         in giftAmounts)
{
    Console.WriteLine(
        amount);
}
```

Use `foreach` when:

```EXAMPLE / FLOW
You need each element
but not the numeric index.
```

Use `for` when:

```EXAMPLE / FLOW
You need the index

You need to move backward

You need custom stepping

You need position-based updates
```

Modifying Using for

```C#
decimal[] amounts = [
    100m,
    200m,
    300m
];

for (int i = 0;
     i < amounts.Length;
     i++)
{
    amounts[i] *= 2m;
}
```

Result:

EXAMPLE / FLOW

```
200
400
600
```

foreach and Modification

For value-type elements:

```C#
foreach (decimal amount
         in amounts)
{
    // amount is the iteration value.
}
```

You cannot normally assign:

```C#
amount = 500m;
```

to replace the array element.

Use index-based access if you need to replace elements.

Reference Objects Inside Arrays

Suppose:

```C#
Gift[] gifts = [
    new Gift
    {
        TotalAmount = 100m
    },

    new Gift
    {
        TotalAmount = 200m
    }
];
```

Then:

```C#
foreach (Gift gift
         in gifts)
{
    gift.TotalAmount += 10m;
}
```

can modify each referenced `Gift` object's state.

Why?

The array contains:

```EXAMPLE / FLOW
references to Gift objects
```

not complete independent immutable objects.

One-Dimensional Arrays

The most common form:

```C#
int[] values = [
    1,
    2,
    3
];
```

This is sometimes called:

```EXAMPLE / FLOW
single-dimensional
```

or:

```EXAMPLE / FLOW
one-dimensional
```

array.

Conceptually:

```EXAMPLE / FLOW
[1][2][3]
```

Multidimensional Arrays

C# supports arrays with multiple dimensions.

Example:

```C#
int[,] matrix = new int[2, 3];
```

Meaning:

EXAMPLE / FLOW

```
2 rows
3 columns
```

Conceptually:

EXAMPLE / FLOW

```
       Col0 Col1 Col2

Row0    0    0    0
Row1    0    0    0
```

Two-Dimensional Initialization

C#

```
int[,] matrix = {
    { 1, 2, 3 },
    { 4, 5, 6 }
};
```

Conceptually:

EXAMPLE / FLOW

```
1 2 3
4 5 6
```

Access:

C#

```
int value = matrix[1, 2];
```

Result:

EXAMPLE / FLOW

```
6
```

Multidimensional Array Syntax

EXAMPLE / FLOW

```
[]
    →
one dimension

[,]
    →
two dimensions

[,,]
    →
three dimensions
```

Example:

C#

```
int[,,] values = new int[2, 3, 4];
```

Rank

An array's:

EXAMPLE / FLOW
```
Rank
```

is the number of dimensions.

Example:

C#
```
int[,] matrix = new int[2, 3];

Console.WriteLine(
    matrix.Rank);
```

Result:

EXAMPLE / FLOW
```
2
```

GetLength

For:

C#
```
int[,] matrix = new int[2, 3];
```

Use:

C#
```
matrix.GetLength(0)
```

Result:

EXAMPLE / FLOW
```
2
```

Use:

C#
```
matrix.GetLength(1)
```

Result:

EXAMPLE / FLOW
```
3
```

Length on Multidimensional Array

C#
```
int[,] matrix = new int[2, 3];
```

Then:

C#
```
matrix.Length
```

is:

EXAMPLE / FLOW
```
6
```

because total number of elements is:

EXAMPLE / FLOW
```
2 × 3
=
6
```

Iterating a Multidimensional Array

C#
```
for (int row = 0;
     row < matrix.GetLength(0);
     row++)
{
    for (int column = 0;
         column < matrix.GetLength(1);
         column++)
    {
        Console.WriteLine(
            matrix[row, column]);
    }
}
```

Flow:

EXAMPLE / FLOW
```
Row 0
    →
Column 0
Column 1
Column 2

Row 1
    →
Column 0
Column 1
Column 2
```

Fundraising Matrix Example

Imagine monthly totals by campaign:

EXAMPLE / FLOW
```
                Jan    Feb    Mar

Annual Giving   1000   1500   2000
Research         500    750    900
```

You could represent it as:

```C#
decimal[,] totals = {
    { 1000m, 1500m, 2000m },
    {  500m,  750m,  900m }
};
```

However, in a real business application, a collection of named objects may often be clearer than an anonymous numeric matrix.

When Multidimensional Arrays Make Sense

Useful for:

```EXAMPLE / FLOW
Matrices

Grid algorithms

Game boards

Image-processing data

Dense mathematical tables

Fixed rectangular structures
```

Less natural for:

```EXAMPLE / FLOW
Business entities with names,
relationships,
and varying row lengths.
```

Jagged Arrays

Definition

A jagged array is:

```EXAMPLE / FLOW
an array whose elements are arrays.
```

Syntax:

```C#
int[][] values;
```

This means:

```EXAMPLE / FLOW
Outer Array
    →
int[] element
int[] element
int[] element
```

Microsoft describes jagged arrays as "arrays of arrays"; each nested array can have a different length.

Jagged Array Example

C#
```
int[][] values = [
    [1, 2],
    [3, 4, 5],
    [6]
];
```

Conceptually:

EXAMPLE / FLOW
```
values[0]
    →
[1,2]

values[1]
    →
[3,4,5]

values[2]
    →
[6]
```

Rows do not need to have equal lengths.

Accessing Jagged Array

C#
```
int value = values[1][2];
```

First:

EXAMPLE / FLOW
```
values[1]
    →
[3,4,5]
```

Then:

EXAMPLE / FLOW
```
[2]
    →
5
```

Result:

EXAMPLE / FLOW
```
5
```

Jagged vs Multidimensional

Multidimensional:

C#
```
int[,] matrix;
```

Concept:

EXAMPLE / FLOW

```
One rectangular array object
```

Jagged:

C#

```
int[][] rows;
```

Concept:

EXAMPLE / FLOW

```
Outer array
    →
separate inner arrays
```

Jagged Array Memory Model

EXAMPLE / FLOW

```
Outer Array
    →
Reference 0 —→ [1,2]
Reference 1 —→ [3,4,5]
Reference 2 —→ [6]
```

The nested arrays are separate array objects.

Why Jagged Arrays Can Be Useful

Suppose campaigns have different numbers of monthly data points:

EXAMPLE / FLOW

```
Campaign A
    →
12 months

Campaign B
    →
6 months

Campaign C
    →
3 months
```

A jagged structure can naturally represent differing row lengths.

Jagged Array Initialization Warning

If:

C#

```
int[][] values = new int[3][];
```

the outer array exists.

But each inner array initially contains:

EXAMPLE / FLOW

```
null
```

because each element is an `int[]` reference.

So:

C#

```
values[0] = new int[2];
```

must create the inner array before accessing its elements.

Implicitly Typed Arrays

Example:

C#

```
var amounts = new[]
    {
        10m,
        20m,
        30m
    };
```

Compiler infers:

EXAMPLE / FLOW

```
decimal[]
```

Type Inference Must Find Compatible Type

Example:

C#

```
var values = new[]
    {
        1,
        2,
        3
    };
```

becomes:

EXAMPLE / FLOW

```
int[]
```

But a set of values without a common implicit element type can fail compilation.

Modern collection-expression target typing can often make intent clearer:

C#

```
decimal[] values = [
    1m,
    2m,
    3m
];
```

Array Copying — Important Distinction

Consider:

C#
```
int[] original = [
    1,
    2,
    3
];

int[] copy = original;
```

This is **not copying the array**.

It is:

EXAMPLE / FLOW
```
copying the reference.
```

Reference Assignment

EXAMPLE / FLOW
```
original ──┐
           ↓
        [1,2,3]
           ↑
copy ──────┘
```

Actual Array Copy

You need an operation that creates another array.

Examples include:

EXAMPLE / FLOW
```
Clone

Array.Copy

range slicing

copying into another destination
```

Clone

C#
```
int[] original = [
    1,
    2,
    3
];

int[] copy = (int[])original.Clone();
```

Now:

EXAMPLE / FLOW

```
original
    →
Array A

copy
    →
Array B
```

`Array.Clone()` performs a **shallow copy** of the array.

Shallow Copy

For value-type elements:

C#

```
int[] original = [
    1,
    2,
    3
];
```

copying elements effectively gives independent integers.

But consider:

C#

```
Gift[] original = [
    gift1,
    gift2
];
```

A shallow copy creates a new array but copies the references:

EXAMPLE / FLOW

```
Original Array ──→ Gift A
               └→ Gift B

Copied Array   ──→ Gift A
               └→ Gift B
```

The arrays are different.

The `Gift` objects are shared.

Microsoft explicitly documents `Clone` as a shallow copy: references are copied, not the referenced objects themselves.

Array.Copy

Example:

C#

```
int[] source = [
    10,
    20,
    30,
    40
];

int[] destination = new int[4];
```

```
Array.Copy(
    source,
    destination,
    source.Length);
```

Result:

```
EXAMPLE / FLOW
destination
=
[10,20,30,40]
```

`Array.Copy` copies a range of elements from one array into another.

Partial Copy

```
C#
Array.Copy(
    source,
    sourceIndex: 1,
    destination,
    destinationIndex: 0,
    length: 2);
```

Copies:

```
EXAMPLE / FLOW
20
30
```

into destination positions:

```
EXAMPLE / FLOW
0
1
```

CopyTo

Arrays also support:

```
C#
source.CopyTo(
    destination,
    index: 0);
```

This copies source elements into an existing destination array starting at the specified destination index.

Deep Copy

Neither:

```
EXAMPLE / FLOW
Clone
```

nor normal:

automatically deep-clones referenced objects.

If each `Gift` should become an independent cloned `Gift`, you need to define how a `Gift` itself should be copied.

Deep copy is a domain/design decision.

Array.Resize

You may see:

```
C#
Array.Resize(
    ref values,
    10);
```

This can look like the original array changed size.

But arrays cannot actually be resized in place.

`Array.Resize` allocates a new one-dimensional array, copies elements, and replaces the caller's array reference.

Resize Flow

Before:

```
EXAMPLE / FLOW
values
    →
Array A
Length 3
```

Call:

```
C#
Array.Resize(
    ref values,
    5);
```

Conceptually:

```
EXAMPLE / FLOW
Allocate Array B
Length 5
    →
Copy old elements
    →
values now points to Array B
```

Old Array A may later be collected if nothing else references it.

Why Array.Resize Uses ref

Because:

EXAMPLE / FLOW
```
the caller's array variable
```

must be changed to point to the new array.

This connects directly to your earlier `ref` topic.

Should You Repeatedly Resize Arrays?

Usually no.

If you need:

EXAMPLE / FLOW
```
frequent adding/removing
```

use:

EXAMPLE / FLOW
```
List<T>
```

instead.

Repeated `Array.Resize` may repeatedly:

EXAMPLE / FLOW
```
allocate
+
copy
```

which is inefficient.

Common System.Array Operations

Useful members include:

EXAMPLE / FLOW
```
Array.Sort

Array.Reverse

Array.IndexOf

Array.BinarySearch

Array.Find

Array.FindIndex

Array.FindAll

Array.Exists

Array.Clear

Array.Copy

Array.Resize

Clone
```

Array.Sort

```C#
decimal[] amounts = [
    500m,
    100m,
    300m
];

Array.Sort(
    amounts);
```

Result:

```EXAMPLE / FLOW
100
300
500
```

Important:

```EXAMPLE / FLOW
Array.Sort
```

modifies the array in place.

Descending Sort

One simple approach:

```C#
Array.Sort(
    amounts);

Array.Reverse(
    amounts);
```

Result:

```EXAMPLE / FLOW
500
300
100
```

Later LINQ will give more expressive projection-based ordering.

Array.Reverse

```C#
int[] values = [
    1,
    2,
    3
];

Array.Reverse(
    values);
```

Result:

EXAMPLE / FLOW

```
3
2
1
```


It modifies the array.

Array.IndexOf

C#

```
int[] values = [
    10,
    20,
    30
];

int index = Array.IndexOf(
        values,
        20);
```

Result:

EXAMPLE / FLOW

```
1
```

If not found:

EXAMPLE / FLOW

```
-1
```

Array.Find

C#

```
decimal[] amounts = [
    100m,
    500m,
    7000m
];

decimal firstMajor = Array.Find(
        amounts,
        amount =>
            amount >= 5000m);
```

Result:

EXAMPLE / FLOW

```
7000
```

This introduces a lambda, which we will study properly later.

Array.Exists

```C#
bool hasMajorGift = Array.Exists(
        amounts,
        amount =>
            amount >= 5000m);
```

Result:

```EXAMPLE / FLOW
true
```

Array.FindAll

```C#
decimal[] majorGifts = Array.FindAll(
        amounts,
        amount =>
            amount >= 5000m);
```

Returns another array containing matching values.

Array.Clear

```C#
int[] values = [
    10,
    20,
    30
];

Array.Clear(
    values);
```

Elements become their default values:

```EXAMPLE / FLOW
0
0
0
```

For reference-type arrays:

```EXAMPLE / FLOW
references become null/default.
```

BinarySearch

When an array is sorted, `Array.BinarySearch` can search efficiently.

Example:

```C#
int[] values = [
    10,
    20,
```

```
    30,
    40,
    50
];

int index = Array.BinarySearch(
        values,
        30);
```

A successful result gives the index.

Important:

EXAMPLE / FLOW

```
Binary search assumes suitable ordering.
```

Do not call it on an arbitrarily unsorted array and expect meaningful results.

Index from End: ^

Modern C# supports:

EXAMPLE / FLOW

```
^
```

for indexes relative to the end.

Example:

C#

```
int[] values = [
    10,
    20,
    30,
    40
];

int last = values[^1];
```

Result:

EXAMPLE / FLOW

```
40
```

Understanding ^

EXAMPLE / FLOW

```
^1
    →
last element

^2
    →
second from last

^3
    →
third from last
```

Why not:

for last?

Because `^0` represents:

EXAMPLE / FLOW
```
the position immediately after the last element
```

when interpreted as an index.

The C# range specification defines from-end indexes relative to the past-end position.

Traditional vs From-End Index

These are equivalent:

C#
```
values[values.Length - 1]
```

and:

C#
```
values[^1]
```

The second is often clearer.

Index Type

The syntax:

C#
```
^1
```

corresponds conceptually to:

EXAMPLE / FLOW
```
System.Index
```

You can explicitly write:

C#
```
Index lastIndex = ^1;

int last = values[lastIndex];
```

Usually direct syntax is simpler.

Ranges

C# uses:

EXAMPLE / FLOW
```
..
```

to describe a range.

Example:

ARRAYS / TUPLES

C#
```
int[] values = [
    10,
    20,
    30,
    40,
    50
];

int[] middle = values[1..4];
```

Result:

EXAMPLE / FLOW
```
20
30
40
```

Important:

EXAMPLE / FLOW
```
start is inclusive

end is exclusive
```

So:

EXAMPLE / FLOW
```
1..4
```

means:

EXAMPLE / FLOW
```
index 1
index 2
index 3
```

not index 4.

Range Flow

EXAMPLE / FLOW
```
values
[10][20][30][40][50]
  0   1   2   3   4

1..4
     ↓
[20][30][40]
```

Open-Ended Range

From beginning:

C#
```
int[] firstThree = values[..3];
```

Result:

EXAMPLE / FLOW

```
10
20
30
```


From index onward:

C#

```
int[] afterFirstTwo = values[2..];
```

Result:

EXAMPLE / FLOW

```
30
40
50
```

Full Range

C#

```
int[] copy = values[..];
```

This selects the entire array.

For arrays, range slicing creates a **new array using a shallow copy** of the selected elements.

Therefore:

C#

```
int[] copy = values[..];
```

is not simply another reference assignment.

Range with From-End

C#

```
int[] lastTwo = values[^2..];
```

Result:

EXAMPLE / FLOW

```
40
50
```

Drop Last Element

C#

```
int[] exceptLast = values[..^1];
```

Result:

EXAMPLE / FLOW

```
10
20
```

```
30
40
```


The language specification defines these shorthand range forms explicitly.

Range Type

You can explicitly declare:

```C#
Range range = 1..4;
```

Then:

```C#
int[] result = values[range];
```

Usually inline syntax is clearer.

Important Range Allocation Rule for Arrays

This:

```C#
int[] result = values[1..4];
```

creates another array.

So repeated slicing:

```C#
for (...)
{
    var slice = values[start..end];
}
```

can create many allocations.

For high-performance code, you may later use:

EXAMPLE / FLOW

```
Span<T>
ReadOnlySpan<T>
```

which can represent slices without copying.

For ordinary business code, ranges are often perfectly acceptable.

Range vs Span

Array range:

```C#
int[] slice = values[1..4];
```

Conceptually:

```
EXAMPLE / FLOW
Allocate New Array
    →
Copy elements
```

Span:

```
C#
Span<int> slice = values.AsSpan(
        1,
        3);
```

Conceptually:

```
EXAMPLE / FLOW
View existing memory
    →
No separate array copy
```

`Span<T>` is covered more deeply later under memory/performance topics.

Array Covariance — Advanced Warning

C# reference-type arrays support covariance.

Example:

```
C#
string[] strings = [
    "A",
    "B"
];

object[] objects = strings;
```

This is allowed because:

```
EXAMPLE / FLOW
string
    →
object
```

is a reference conversion.

But now:

```
C#
objects[0] = new Gift();
```

compiles because `objects` has type:

```
EXAMPLE / FLOW
object[]
```

but fails at runtime because the actual array is still:

```
EXAMPLE / FLOW
string[]
```

This can produce:

EXAMPLE / FLOW

```
ArrayTypeMismatchException
```

Why Array Covariance Can Be Dangerous

Runtime object:

EXAMPLE / FLOW

```
string[]
```

Compile-time variable:

EXAMPLE / FLOW

```
object[]
```

The runtime must ensure only strings enter the actual array.

This is one reason generic collections such as:

EXAMPLE / FLOW

```
List<T>
```

do not simply behave with the same unsafe array covariance model.

When Should You Use an Array?

Use an array when:

EXAMPLE / FLOW

```
Element count is fixed or naturally known

You want indexed sequential data

You receive an array from an API

Performance/data layout matters

A fixed batch is appropriate

Interop/API contract requires array
```

When Not to Use Array

Avoid arrays as the default when:

EXAMPLE / FLOW

```
Items frequently added

Items frequently removed

Size changes regularly

Rich collection operations required
```

In those situations:

EXAMPLE / FLOW

```
List<T>

HashSet<T>

Dictionary<TKey,TValue>
```

may be better.

These are coming later.

Tuples

Now we move from:

EXAMPLE / FLOW

```
many values of the same type
```

to:

EXAMPLE / FLOW

```
a small number of related values,
possibly of different types.
```

What Is a Tuple?

Definition

A tuple is a lightweight structure that groups multiple values together.

Example:

C#

```
(long, decimal) gift = (
        1001,
        500m
    );
```

This groups:

EXAMPLE / FLOW

```
GiftId
+
Amount
```

into one value.

Modern C# tuple syntax is based on:

EXAMPLE / FLOW

```
System.ValueTuple
```

and is a value type. Microsoft describes tuples as lightweight data structures for grouping multiple data elements.

Unnamed Tuple

```C#
(long, decimal) gift = (
        1001,
        500m
    );
```

Access:

```C#
gift.Item1
```

and:

```C#
gift.Item2
```

Result:

EXAMPLE / FLOW

```
Item1
    →
1001

Item2
    →
500
```

This works, but names are usually clearer.

Named Tuple

```C#
(long GiftId, decimal Amount)
    gift = (
        1001,
        500m
    );
```

Now:

```C#
gift.GiftId
```

and:

```C#
gift.Amount
```

are much easier to understand.

Tuple Type and Value

Type:

```
C#
(long GiftId, decimal Amount)
```

Value:

```
C#
(
    1001,
    500m
)
```

Complete:

```
C#
(long GiftId, decimal Amount)
    gift = (
        1001,
        500m
    );
```

Inferred Tuple Names

Example:

```
C#
long giftId = 1001;

decimal amount = 500m;

var gift = (
        giftId,
        amount
    );
```

The compiler can infer tuple element names:

```
EXAMPLE / FLOW
gift.giftId
gift.amount
```

Explicit names are often clearer when designing APIs.

Tuple Method Return

This is one of the best uses of tuples.

```
C#
static (
    int Count,
    decimal Total)
    CalculateSummary(
        decimal[] amounts)
{
    int count = amounts.Length;

    decimal total = 0m;

    foreach (decimal amount
             in amounts)
```

```
    {
        total += amount;
    }

    return (
        count,
        total
    );
}
```

Call:

```C#
var summary = CalculateSummary(
        amounts);
```

Use:

```C#
summary.Count
summary.Total
```

Why Tuple Return Can Be Better Than out

Old-style:

```C#
void CalculateSummary(
    decimal[] amounts,
    out int count,
    out decimal total)
```

Tuple:

```C#
(int Count, decimal Total)
    CalculateSummary(...)
```

The tuple makes output values part of the return value.

This is usually easier to understand.

Tuple Deconstruction

You can extract elements into variables.

```C#
var summary = (
        Count: 10,
        Total: 5000m
    );

var (
    count,
    total
) = summary;
```

Now:

EXAMPLE / FLOW

```
count = 10
total = 5000
```

Tuple types support direct deconstruction.

Direct Deconstruction from Method

C#

```
var (
    count,
    total
) =
    CalculateSummary(
        amounts);
```

No intermediate tuple variable required.

Explicit Types in Deconstruction

C#

```
(
    int count,
    decimal total
) =
    CalculateSummary(
        amounts);
```

Discards

If you don't need one tuple value:

C#

```
var (
    _,
    total
) =
    CalculateSummary(
        amounts);
```

`_` means:

EXAMPLE / FLOW

```
discard this value.
```

Microsoft documents discards as the standard way to ignore unwanted tuple elements during deconstruction.

Multiple Discards

C#

```
var (
    count,
    _,
    _
) = result;
```

Only:

EXAMPLE / FLOW

```
count
```

ARRAYS / TUPLES

is retained.

Tuple Assignment

Tuples with compatible element positions/types can be assigned even if their element names differ.

Example:

C#

```
(int Min, int Max)
    first = (
        1,
        10
    );

(int Start, int End)
    second = first;
```

Values are matched by:

EXAMPLE / FLOW

```
position
```

not by tuple element name.

Microsoft notes that tuple assignment depends on arity and compatible element types; element names don't determine assignment compatibility.

Tuple Names Are Mostly Metadata

These:

C#

```
(int Min, int Max)
```

and:

C#

```
(int Start, int End)
```

have the same underlying tuple shape when element types and positions match.

Names improve readability but do not create a brand-new strongly distinct domain type.

Important Tuple Limitation

Suppose you define:

C#

```
using GiftRange = (
        decimal Minimum,
        decimal Maximum
    );
```

This creates an alias.

It does **not** create a new type.

Microsoft explicitly notes that a tuple alias is a synonym, not a new type.

If type safety/business identity matters, use:

```
EXAMPLE / FLOW
record
struct
class
```

instead.

Tuple Equality

Tuples support value equality.

Example:

```
C#
var first = (
        Id: 1001,
        Amount: 500m
    );

var second = (
        Id: 1001,
        Amount: 500m
    );

bool equal = first == second;
```

Result:

```
EXAMPLE / FLOW
true
```

Tuple element values are compared positionally.

Tuple Names Do Not Affect Equality

Example concept:

```
EXAMPLE / FLOW
(Id: 1001, Amount: 500)
```

and:

```
EXAMPLE / FLOW
(GiftId: 1001, Total: 500)
```

can still compare based on corresponding values/types rather than the names themselves.

Tuples Are Value Types

This matters.

```C#
var first = (
        Count: 5,
        Total: 100m
    );

var second = first;
```

The tuple value is copied.

Changing `second` does not automatically mutate `first`.

Tuple Containing Reference Types

But remember Topic 6.

A tuple can contain references:

```C#
var first = (
        Gift: gift,
        Label: "Current"
    );
```

Copying the tuple copies:

```EXAMPLE / FLOW
the Gift reference
```

not the actual `Gift` object.

So both tuple copies may refer to the same mutable `Gift`.

When to Use Tuples

Good for:

```EXAMPLE / FLOW
Small method results

Temporary internal grouping

Local transformations

Short-lived calculations

Two or three naturally related values
```

Example:

```EXAMPLE / FLOW
Count + Total
```

When Not to Use Tuples

Avoid using huge tuples:

```C#
(
    long,
```

```
    long,
    decimal,
    string,
    DateTime,
    bool,
    string,
    int
)
```

for an important business concept.

A named type is clearer:

```C#
public sealed record GiftSummary(...);
```

Tuple vs Business DTO

Tuple:

```C#
(long GiftId, decimal Amount)
```

Good for:

```EXAMPLE / FLOW
private/internal short-lived calculation
```

DTO:

```C#
public sealed record GiftResponse(
    long GiftId,
    decimal Amount);
```

Better for:

```EXAMPLE / FLOW
API contracts

cross-layer communication

serialization

long-lived domain/application models

public library APIs
```

KeyValuePair<TKey,TValue>

Definition

`KeyValuePair<TKey,TValue>` represents:

```EXAMPLE / FLOW
one key
+
one associated value
```

Example:

C#
```
var pair = new KeyValuePair<long, decimal>(
        101,
        500m);
```

Here:

EXAMPLE / FLOW
```
Key
    →
101

Value
    →
500
```

Common Use

You usually encounter `KeyValuePair<TKey,TValue>` while iterating:

EXAMPLE / FLOW
```
Dictionary<TKey,TValue>
```

Example later:

C#
```
foreach (
    KeyValuePair<long, decimal> pair
    in campaignTotals)
{
    Console.WriteLine(
        pair.Key);

    Console.WriteLine(
        pair.Value);
}
```

Fundraising Key/Value Example

Conceptually:

EXAMPLE / FLOW
```
CampaignId
    →
Total Raised
```

could appear as:

C#
```
KeyValuePair<long, decimal>
```

where:

EXAMPLE / FLOW
```
Key
=
CampaignId
```

```
Value
=
TotalRaised
```


KeyValuePair Is a Struct

The generic:

```C#
KeyValuePair<TKey,TValue>
```

type is a value-type structure.

For example, dictionary enumeration produces these pair values.

Creating a Pair

Traditional:

```C#
var pair = new KeyValuePair<long, decimal>(
        1001,
        500m);
```

Modern .NET also provides a helper:

```C#
var pair = KeyValuePair.Create(
        1001L,
        500m);
```

The `KeyValuePair` helper provides `Create<TKey,TValue>` for constructing generic pairs.

Deconstructing KeyValuePair

Modern .NET supports deconstruction of key/value pairs.

Example:

```C#
foreach (
    var (
        campaignId,
        total
    )
    in campaignTotals)
{
    Console.WriteLine(
        $"{campaignId}: {total}");
}
```

Microsoft documents `KeyValuePair<TKey,TValue>` as supporting deconstruction in dictionary iteration.

KeyValuePair vs Tuple

Tuple:

```C#
(long CampaignId, decimal Total)
```

provides meaningful names.

KeyValuePair:

```C#
KeyValuePair<long, decimal>
```

specifically communicates:

```
EXAMPLE / FLOW
Key
→
Value
```

Use `KeyValuePair` when key/value semantics matter.

Use tuple when the relationship is not specifically a key lookup pair.

Anonymous Types

Definition

An anonymous type is a compiler-generated type that lets you group named read-only properties without explicitly declaring a class.

Example:

```C#
var gift = new
    {
        GiftId = 1001,

        Amount = 500m
    };
```

The compiler creates an anonymous class type for you.

Microsoft describes anonymous types as compiler-generated types with read-only properties that don't have a source-code name you can explicitly use.

Why Anonymous Types Exist

Imagine you only need:

```
EXAMPLE / FLOW
GiftId
DonorName
Amount
```

for one local query.

Creating:

```C#
public class TemporaryGiftThing
```

may be unnecessary.

Instead:

```C#
var result = new
    {
        GiftId = gift.GiftId,

        Amount = gift.TotalAmount
    };
```

Anonymous Type Property Inference

Suppose:

```C#
long giftId = 1001;

decimal amount = 500m;
```

You can write:

```C#
var result = new
    {
        giftId,
        amount
    };
```

Compiler infers property names:

```EXAMPLE / FLOW
giftId

amount
```

Microsoft supports inferred property names from variables/member access expressions.

Why var Is Required

You cannot write:

```EXAMPLE / FLOW
SomeAnonymousType x = ...
```

because there is no source-level name you can declare.

Therefore:

```C#
var result = new
    {
        GiftId = 1001
    };
```

is required.

Anonymous Properties Are Read-Only

Example:

```C#
var result = new
    {
        GiftId = 1001,

        Amount = 500m
    };
```

This is not normally mutable:

```C#
// result.Amount = 1000m;
```

The generated properties are read-only.

Anonymous Type and LINQ

This is where anonymous types are very common.

Later in LINQ you will write:

```C#
var results = gifts.Select(
        gift =>
            new
            {
                gift.GiftId,
                gift.TotalAmount
            });
```

This projects each `Gift` into a lightweight temporary shape.

Example Projection

Source:

```EXAMPLE / FLOW
Gift
    →
GiftId
ConstituentId
CampaignId
Amount
CreatedAt
Notes
...
```

Query needs only:

```EXAMPLE / FLOW
GiftId
Amount
```

Anonymous projection:

```EXAMPLE / FLOW
Gift
    →
{
    GiftId,
```

```
    Amount
};
```


This can avoid carrying unnecessary fields around in local query logic.

Anonymous Type Equality

Anonymous types with the same compatible property structure generated in the same assembly receive compiler-generated value-style equality behavior.

Microsoft's current documentation states that compiler-generated anonymous types implement `Equals` and `GetHashCode` based on property values.

Anonymous Type Limitations

You cannot conveniently use an anonymous type as:

```
EXAMPLE / FLOW
Public API contract

Named method parameter type

Normal named field type

Long-lived domain model
```

because you cannot write its generated type name directly.

Microsoft specifically lists method return/parameter/field usage as limitations because anonymous types can't be named from source code.

Tuple vs Anonymous Type

Tuple:

```
C#
var result = (
        GiftId: 1001,
        Amount: 500m
    );
```

Anonymous:

```
C#
var result = new
    {
        GiftId = 1001,

        Amount = 500m
    };
```

Major Difference

Tuple:

```
EXAMPLE / FLOW
Value type

Easy deconstruction
```

```
Good small return value
```

Anonymous type:

```
EXAMPLE / FLOW
Compiler-generated class

Named read-only properties

Excellent LINQ projection
```

When to Use Anonymous Types

Good:

```
EXAMPLE / FLOW
LINQ projections

Temporary local shaping

Short-lived query data
```

Avoid:

```
EXAMPLE / FLOW
Domain models

API contracts

Service interfaces

Repository contracts
```

Use a named DTO/record instead.

Arrays vs Lists — Preview

Array:

```
EXAMPLE / FLOW
Fixed length

Fast indexed access

Lower-level representation

Good when size known
```

List:

```
EXAMPLE / FLOW
Dynamic count

Add/remove

Convenient collection API
```

Example:

```C#
int[] values = new int[3];
```

vs:

```C#
List<int> values = new();
```

We will study `List<T>` properly in Topic 21.

Array Performance

Array indexing is very efficient.

Example:

```C#
decimal amount = amounts[500];
```

is conceptually constant-time indexed access:

```
EXAMPLE / FLOW
O(1)
```

assuming a valid index.

Iterating an Array

Processing every element requires:

```
EXAMPLE / FLOW
O(n)
```

because each element must be visited.

Example:

```C#
foreach (decimal amount
         in amounts)
{
}
```

For `n` elements:

```
EXAMPLE / FLOW
n iterations
```

Searching Unsorted Array

Linear search:

```C#
Array.IndexOf(...)
```

may need to examine many elements.

Conceptually:

EXAMPLE / FLOW

```
O(n)
```

Sorting

General comparison sorting is typically on the order of:

EXAMPLE / FLOW

```
O(n log n)
```

for normal array sorting algorithms, though implementation details can vary by element type/comparer/runtime.

For normal business code, focus on:

EXAMPLE / FLOW

```
correct data structure
```

before micro-optimizing array syntax.

Arrays and Memory

An array stores a fixed number of elements.

For value-type arrays:

C#

```
decimal[] amounts;
```

the values are stored in the array's element storage.

For reference-type arrays:

C#

```
Gift[] gifts;
```

the array stores references to `Gift` objects.

Large Arrays

Very large arrays consume contiguous managed storage for their element data.

Therefore avoid casually allocating:

C#

```
new byte[2_000_000_000];
```

or similarly enormous arrays.

High-volume processing may require:

EXAMPLE / FLOW

```
Streaming

Batching

Pooling

Span/Memory
```

```
Incremental processing
```


depending on the use case.

Security: Trusting Array Indexes

If an index comes from external/user input:

```C#
values[userIndex]
```

validate the range.

Example:

```C#
if (userIndex < 0
    ||
    userIndex >= values.Length)
{
    throw new ArgumentOutOfRangeException(
        nameof(userIndex));
}
```

Security: Sensitive Arrays

Arrays may contain sensitive bytes:

```C#
byte[] secret;
```

Simply losing a reference does not guarantee immediate memory erasure.

Security-sensitive cryptographic APIs may require explicit clearing or specialized types.

Do not store secrets in ordinary long-lived global/static arrays unnecessarily.

Common Mistake — Using <= Length

Wrong:

```C#
for (int i = 0;
     i <= values.Length;
     i++)
```

Correct:

```C#
for (int i = 0;
     i < values.Length;
     i++)
```

Common Mistake — Expecting Array to Grow

```C#
int[] values = new int[3];
```

You cannot simply:

EXAMPLE / FLOW
```
Add fourth slot
```

to the same array instance.

Choose `List<T>` if dynamic growth is fundamental.

Common Mistake — Assignment vs Copy

Wrong expectation:

C#
```
int[] copy = original;
```

does **not** create an independent array.

It copies the reference.

Common Mistake — Clone Means Deep Clone

C#
```
Gift[] copy = (Gift[])original.Clone();
```

does not clone each `Gift`.

It copies the references.

Common Mistake — Array.Resize Changes Same Array

It does not.

`Array.Resize` allocates another array, copies values, then changes the caller's variable to reference it.

Common Mistake — Multidimensional vs Jagged

These are different:

C#
```
int[,]
```

and:

C#
```
int[][]
```

`int[,]`:

EXAMPLE / FLOW
```
rectangular multidimensional array
```

`int[][]`:

EXAMPLE / FLOW
```
array of separate arrays
```

Common Mistake — Range End Is Inclusive

Wrong assumption:

EXAMPLE / FLOW

```
1..4
```

does **not** include index 4.

Correct:

EXAMPLE / FLOW

```
start inclusive
end exclusive
```

Common Mistake — Array Slice Is Just a View

For arrays:

C#

```
var slice = values[1..4];
```

creates a new array containing a shallow copy of the selected elements.

It is not simply an allocation-free view.

Common Mistake — Too Much Business Meaning in Tuple

Avoid:

C#

```
(
    long,
    long,
    long,
    decimal,
    DateTime,
    bool,
    string
)
```

for important application data.

Create a proper type.

Common Mistake — Anonymous Type Across Layers

Do not make anonymous types the basis of:

EXAMPLE / FLOW

```
Controller → Service

Service → Repository

Public API contract
```

because the generated type has no normal source-level name.

Use DTOs/records.

Practical Coding Problem — Reverse an Array Manually

Requirement:

EXAMPLE / FLOW

```
Reverse without Array.Reverse.
```

Example:

C#

```
static void Reverse(
    int[] values)
{
    int left = 0;

    int right = values.Length - 1;

    while (left < right)
    {
        int temporary = values[left];

        values[left] = values[right];

        values[right] = temporary;

        left++;
        right--;
    }
}
```

Input:

EXAMPLE / FLOW

```
1 2 3 4 5
```

Flow:

EXAMPLE / FLOW

```
Swap 1 ↔ 5
    →
5 2 3 4 1

Swap 2 ↔ 4
    →
5 4 3 2 1

Pointers meet
    →
stop
```

Result:

EXAMPLE / FLOW

```
5 4 3 2 1
```

Why This Algorithm Is Good

Time:

```
O(n)
```

Extra space:


```
O(1)
```

because it reverses the existing array in place.

Practical Coding Problem — Find Duplicates

Input:

C#

```
int[] values = [
    1,
    2,
    3,
    2,
    4,
    1
];
```

Beginner approach:

C#

```
static void PrintDuplicates(
    int[] values)
{
    for (int i = 0;
         i < values.Length;
         i++)
    {
        bool alreadyPrinted = false;

        for (int previous = 0;
             previous < i;
             previous++)
        {
            if (values[previous]
                == values[i])
            {
                alreadyPrinted = true;

                break;
            }
        }

        if (alreadyPrinted)
        {
            continue;
        }

        for (int j = i + 1;
             j < values.Length;
             j++)
        {
            if (values[i]
                == values[j])
            {
```

```
                Console.WriteLine(
                    values[i]);

                break;
            }
        }
    }
}
```

This demonstrates array fundamentals but can be:

EXAMPLE / FLOW

$O(n^2)$

Later `HashSet<T>` gives a more efficient solution.

Duplicate Detection with Sorting

Another approach:

C#

```
static void PrintDuplicates(
    int[] values)
{
    int[] copy = (int[])values.Clone();

    Array.Sort(
        copy);

    for (int i = 1;
         i < copy.Length;
         i++)
    {
        if (copy[i]
            == copy[i - 1])
        {
            if (i == 1
                ||
                copy[i]
                != copy[i - 2])
            {
                Console.WriteLine(
                    copy[i]);
            }
        }
    }
}
```

Sorting changes complexity and avoids modifying the original by cloning first.

Practical Coding Problem — Second Largest Distinct Value

C#

```
static int FindSecondLargest(
    int[] values)
{
    if (values.Length < 2)
    {
        throw new ArgumentException(
            "At least two values are required.",
            nameof(values));
```

```
    }

    int? largest = null;

    int? secondLargest = null;

    foreach (int value
             in values)
    {
        if (largest is null
            ||
            value > largest)
        {
            if (largest is not null)
            {
                secondLargest = largest;
            }

            largest = value;
        }
        else if (
            value != largest
            &&
            (
                secondLargest is null
                ||
                value > secondLargest
            ))
        {
            secondLargest = value;
        }
    }

    return secondLargest
        ??
        throw new InvalidOperationException(
            "No second distinct largest value exists.");
}
```

Example:

EXAMPLE / FLOW

```
10
50
20
50
40
```

Distinct values:

EXAMPLE / FLOW

```
50
40
20
10
```

Result:

EXAMPLE / FLOW

```
40
```

Why Distinct Matters

For:

EXAMPLE / FLOW

```
10
50
50
20
```

Second **largest element** might be interpreted differently from:

EXAMPLE / FLOW

```
second largest distinct value.
```

Always clarify the requirement in interview coding.

Professional coding starts with:

EXAMPLE / FLOW

```
What exactly does "second largest" mean?
```

Practical Coding Problem — Frequency Count

Input:

C#

```
int[] values = [
    1,
    2,
    1,
    3,
    2,
    1
];
```

Beginner array-only approach:

C#

```
static void PrintFrequency(
    int[] values)
{
    bool[] counted = new bool[
            values.Length];

    for (int i = 0;
         i < values.Length;
         i++)
    {
        if (counted[i])
        {
            continue;
        }

        int count = 1;

        for (int j = i + 1;
             j < values.Length;
             j++)
        {
            if (values[i]
```

```
                == values[j])
            {
                count++;

                counted[j] = true;
            }
        }

        Console.WriteLine(
            $"{values[i]} → {count}");
    }
}
```

Output:

EXAMPLE / FLOW

```
1 → 3
2 → 2
3 → 1
```

Later dictionary-based counting will be much cleaner and usually more efficient.

Practical Coding Problem — Remove Duplicates Without LINQ

Input:

EXAMPLE / FLOW

```
1 2 1 3 2 4
```

One beginner approach:

C#

```
static int[] RemoveDuplicates(
    int[] values)
{
    int[] temporary = new int[
            values.Length];

    int uniqueCount = 0;

    foreach (int value
             in values)
    {
        bool exists = false;

        for (int i = 0;
             i < uniqueCount;
             i++)
        {
            if (temporary[i]
                == value)
            {
                exists = true;

                break;
            }
        }

        if (!exists)
        {
            temporary[
```

```
                uniqueCount] = value;

            uniqueCount++;
        }
    }

    int[] result = new int[
            uniqueCount];

    Array.Copy(
        temporary,
        result,
        uniqueCount);

    return result;
}
```

Result:

EXAMPLE / FLOW

```
1
2
3
4
```

Later:

EXAMPLE / FLOW

```
HashSet<T>
Distinct()
```

will make this much easier.

But this version helps you understand the underlying mechanics.

Practical Coding Problem — Find Maximum

C#

```
static decimal FindMaximum(
    decimal[] values)
{
    if (values.Length == 0)
    {
        throw new ArgumentException(
            "Array cannot be empty.",
            nameof(values));
    }

    decimal maximum = values[0];

    for (int i = 1;
         i < values.Length;
         i++)
    {
        if (values[i]
            >
            maximum)
        {
            maximum = values[i];
        }
    }
```

```
    return maximum;
}
```


Practical Coding Problem — Sum

C#

```
static decimal Sum(
    decimal[] values)
{
    decimal total = 0m;

    foreach (decimal value
             in values)
    {
        total += value;
    }

    return total;
}
```

Time:

EXAMPLE / FLOW

```
O(n)
```

Practical Coding Problem — Copy a Range

Using range syntax:

C#

```
int[] values = [
    10,
    20,
    30,
    40,
    50
];

int[] middle = values[1..4];
```

Result:

EXAMPLE / FLOW

```
20
30
40
```

Practical Coding Problem — Last Three

C#

```
int[] lastThree = values[^3..];
```

Result:

EXAMPLE / FLOW

```
30
40
50
```

Fundraising Example — Gift Batch

C#

```
long[] giftIds = [
    1001,
    1002,
    1003,
    1004
];
```

Flow:

EXAMPLE / FLOW

```
Batch
    →
Gift 1001
Gift 1002
Gift 1003
Gift 1004
```

Arrays work well when:

EXAMPLE / FLOW

```
the batch size/value set is already known.
```

Fundraising Example — Gift Summary Tuple

C#

```
static (
    int GiftCount,
    decimal TotalAmount,
    decimal AverageAmount)
    CalculateGiftSummary(
        decimal[] amounts)
{
    if (amounts.Length == 0)
    {
        return (
            GiftCount: 0,
            TotalAmount: 0m,
            AverageAmount: 0m
        );
    }

    decimal total = 0m;

    foreach (decimal amount
             in amounts)
    {
        total += amount;
    }

    return (
        GiftCount:
```

```
            amounts.Length,

        TotalAmount:
            total,

        AverageAmount:
            total / amounts.Length
    );
}
```

Usage:

```
C#
var (
    count,
    total,
    average
) =
    CalculateGiftSummary(
        giftAmounts);
```

Fundraising Example — Anonymous Projection

Later with LINQ:

```
C#
var summaries = gifts.Select(
        gift =>
            new
            {
                gift.GiftId,
                gift.TotalAmount
            });
```

Conceptual transformation:

```
EXAMPLE / FLOW
Full Gift Entity
    →
Temporary Projection
    →
GiftId
Amount
```

This is a common pattern in reporting and querying.

Fundraising Example — Monthly Matrix

```
C#
decimal[,] campaignTotals = {
    {
        1000m,
        1500m,
        1800m
    },

    {
        500m,
        900m,
        1200m
```

```
    }
};
```


Conceptually:

EXAMPLE / FLOW

```
            Jan     Feb     Mar

Campaign A  1000    1500    1800
Campaign B   500     900    1200
```

Fundraising Example — Variable Allocations

Suppose each gift has a different number of allocations:

C#

```
decimal[][] allocations = [
    [
        600m,
        400m
    ],

    [
        1000m
    ],

    [
        500m,
        250m,
        250m
    ]
];
```

This is naturally jagged:

EXAMPLE / FLOW

```
Gift 1
    →
2 allocations

Gift 2
    →
1 allocation

Gift 3
    →
3 allocations
```

Arrays in ASP.NET Core

ASP.NET Core DTOs can contain arrays.

Example:

C#

```
public sealed class GiftBatchRequest
{
    public long[] GiftIds
    {
        get;
        init;
```

```
    } = [];
}
```

Possible JSON:

```json
{
  "giftIds": [
    1001,
    1002,
    1003
  ]
}
```

Should API DTOs Use Arrays or Lists?

Both are possible.

Use array when:

```
Fixed sequential payload
No mutation semantics needed
Simple serialization contract
```

Use `List<T>` when application code naturally:

```
adds/removes/builds items dynamically.
```

Do not choose based only on serialization appearance because JSON arrays map naturally to multiple .NET collection types.

Arrays and Defensive Copying

Suppose:

```csharp
public sealed class Batch
{
    private readonly long[]
        _giftIds;

    public Batch(
        long[] giftIds)
    {
        _giftIds = giftIds;
    }
}
```

Problem:

```csharp
long[] source = [
    1,
    2
];

var batch = new Batch(
        source);
```

```
source[0] = 999;
```


If `Batch` kept the same array reference, its internal data also changed.

Defensive Copy

```C#
public Batch(
    long[] giftIds)
{
    _giftIds = giftIds[..];
}
```

Now the object stores another array.

For value-type IDs, that provides an independent copy of the element values.

This can help preserve encapsulation.

Reference Element Warning

For:

```C#
Gift[] gifts;
```

the defensive array copy:

```C#
_gifts = gifts[..];
```

creates a new array but still shares the individual `Gift` references.

Again:

EXAMPLE / FLOW

```
shallow copy
```

not deep copy.

Interview Questions

What is an array in C#?

An array is a fixed-size indexed sequence containing elements of one element type.

The array itself is a reference type, derives from `System.Array`, and uses zero-based indexing.

Are arrays value types or reference types?

Arrays are reference types.

Assignment:

```C#
int[] second = first;
```

normally copies the array reference rather than all elements.

Are array sizes dynamic?

No.

An array instance's dimensions are fixed when created and cannot later be resized in place.

Then how does Array.Resize work?

It allocates a new one-dimensional array, copies the existing elements as appropriate, and changes the caller's variable to reference the new array.

What are the default values in an array?

Each element receives:

EXAMPLE / FLOW
```
default(T)
```

for its element type.

Examples:

EXAMPLE / FLOW
```
int → 0

bool → false

reference → null
```

What does Length return?

`Length` returns the total number of elements in an array.

For a `2 × 3` multidimensional array:

EXAMPLE / FLOW
```
Length = 6
```

What is Rank?

`Rank` is the number of array dimensions.

C#
```
int[,]
```

has rank:

EXAMPLE / FLOW
```
2
```

What does GetLength(dimension) do?

It returns the number of elements in a specified dimension.

Example:

C#
```
matrix.GetLength(0)
```

returns row count for a typical 2D matrix.

What is a multidimensional array?

An array with more than one dimension.

Example:

```
C#
int[,]
```

represents a rectangular two-dimensional array.

What is a jagged array?

An array whose elements are arrays.

Example:

```
C#
int[][]
```

Each inner array can have a different length.

What is the difference between int[,] and int[][]?

`int[,]`:

```
EXAMPLE / FLOW
One rectangular multidimensional array.
```

`int[][]`:

```
EXAMPLE / FLOW
Outer array containing references
to separate int[] arrays.
```

What does Array.Clone do?

It creates a shallow copy of the array.

A new array is created, but reference-type elements still point to the same underlying objects.

What does Array.Copy do?

It copies a specified number/range of elements from one array into another array.

What is a shallow copy?

A new outer container is created, but nested referenced objects are shared.

Example:

```
EXAMPLE / FLOW
Original Array —→ Gift A
Copied Array   —→ Gift A
```

What is a deep copy?

The outer object and required nested referenced objects are independently copied.

C# arrays do not automatically deep-copy arbitrary objects for you.

What does ^ mean in indexing?

It represents an index relative to the end.

```C#
values[^1]
```

means:

```
EXAMPLE / FLOW
last element
```

What does .. mean?

It represents a range.

Example:

```C#
values[1..4]
```

selects:

```
EXAMPLE / FLOW
indexes 1,2,3
```

because the ending index is exclusive.

Does array slicing allocate?

For normal arrays, using a `Range` returns a new array containing a shallow copy of the selected elements.

What is a tuple?

A tuple is a lightweight value type that groups multiple values together.

Example:

```C#
(long GiftId, decimal Amount)
```

Are modern C# tuples value types?

Yes.

C# tuple syntax uses `System.ValueTuple` structures.

Why use named tuple elements?

This:

```C#
(long GiftId, decimal Amount)
```

is clearer than:

```C#
(long, decimal)
```

because callers can access:

EXAMPLE / FLOW

```
GiftId
Amount
```

instead of:

EXAMPLE / FLOW

```
Item1
Item2
```

What is tuple deconstruction?

It extracts tuple elements into individual variables.

Example:

C#

```
var (
    count,
    total
) = summary;
```

What is a discard?

The underscore:

C#

```
_
```

can ignore a tuple element during deconstruction.

Example:

C#

```
var (
    _,
    total
) = summary;
```

Do tuple element names create a new type?

No.

Names improve readability but do not create distinct nominal types.

Tuple assignment primarily depends on compatible element count and types.

When should a tuple not be used?

Avoid tuples for important long-lived business contracts containing many values.

Use:

EXAMPLE / FLOW

```
class

record

struct
```

```
DTO
```

instead.

What is KeyValuePair<TKey,TValue>?

A value-type pair containing:

EXAMPLE / FLOW

```
Key
+
Value
```

It is commonly encountered during dictionary enumeration.

Can KeyValuePair be deconstructed?

Yes.

Example:

C#

```
foreach (
    var (
        key,
        value
    )
    in dictionary)
```

What is an anonymous type?

A compiler-generated reference type containing named read-only properties.

Example:

C#

```
var result = new
    {
        GiftId = 1001,
        Amount = 500m
    };
```

Why must anonymous types use var?

Because their generated class name isn't accessible as a normal source-code type name.

Where are anonymous types commonly used?

Especially in:

EXAMPLE / FLOW

```
LINQ projections

Temporary query results

Local data shaping
```

Can anonymous types be used as public API DTOs?

They are not appropriate for normal public API contracts because they don't provide a normal named type you can expose and maintain.

Use a named DTO or record.

Best Practices

Use arrays when the number of elements is naturally fixed or already known.

Use:

```
C#
Length
```

instead of hard-coded array bounds.

Use:

```
EXAMPLE / FLOW
i < array.Length
```

not:

```
EXAMPLE / FLOW
i <= array.Length
```

for normal zero-based iteration.

Prefer `foreach` when the index is unnecessary.

Use `for` when position/index behavior matters.

Remember:

```
EXAMPLE / FLOW
Array Assignment
≠
Array Copy
```

Use an explicit copy when independent array storage is required.

Remember that ordinary array copies are generally shallow.

Use multidimensional arrays for naturally rectangular data.

Use jagged arrays when rows naturally have different sizes.

Do not repeatedly use `Array.Resize` when dynamic growth is fundamental; prefer `List<T>`.

Use `^1` when it clearly communicates “last element.”

Remember that a range end is exclusive.

Remember that array range slicing creates another array.

Use tuples for small lightweight method results.

Use meaningful tuple names.

Do not turn major business contracts into large tuples.

Use `KeyValuePair<TKey,TValue>` when the meaning is specifically key/value.

Use anonymous types for local temporary projections, especially with LINQ.

Use named DTOs/records for service, domain, and API boundaries.

Prefer readable ordinary array code before introducing `Span<T>` optimization.

Profile before optimizing array slicing/copying.

Validate external indexes before indexing arrays.

Be cautious with mutable reference objects stored inside arrays because the array stores their references.

Quick Revision

Concept	Meaning
`T[]`	One-dimensional array
`T[,]`	Rectangular two-dimensional array
`T[][]`	Jagged array
`Length`	Total number of elements
`Rank`	Number of dimensions
`GetLength(n)`	Size of dimension `n`
`array[i]`	Access element at index
`^1`	Last element
`a..b`	Range from `a` inclusive to `b` exclusive
`[..]`	Full range copy for array
`Array.Sort`	Sort array in place
`Array.Reverse`	Reverse array
`Array.IndexOf`	Find element index
`Array.Find`	Find matching value
`Array.Exists`	Test whether match exists
`Array.Clear`	Set elements to defaults
`Clone`	Shallow array copy
`Array.Copy`	Copy elements
`Array.Resize`	Allocate replacement array
Tuple	Lightweight grouped values
Named tuple	Tuple with readable element names
Deconstruction	Extract grouped components
`_`	Discard
`KeyValuePair<K,V>`	Key/value pair structure
Anonymous type	Compiler-generated temporary read-only projection
Collection expression	Modern `[a, b, c]` syntax
Spread `..source`	Inline elements into collection expression

Complete Fundraising Example

C#

```
decimal[] giftAmounts = [
    100m,
    250m,
    500m,
    1000m,
    7500m
];

static (
    int Count,
    decimal Total,
    decimal Largest)
    CalculateSummary(
        decimal[] amounts)
{
    if (amounts.Length == 0)
    {
        return (
            0,
            0m,
            0m
        );
    }

    decimal total = 0m;

    decimal largest = amounts[0];

    foreach (decimal amount
             in amounts)
    {
        total += amount;

        if (amount > largest)
        {
            largest = amount;
        }
    }

    return (
        Count:
            amounts.Length,

        Total:
            total,

        Largest:
            largest
    );
}

var (
    giftCount,
    totalGiving,
    largestGift
) =
    CalculateSummary(
        giftAmounts);

decimal[] firstThree = giftAmounts[..3];
```

```
decimal[] lastTwo = giftAmounts[^2..];

Console.WriteLine(
    $"Gift Count: {giftCount}");

Console.WriteLine(
    $"Total Giving: {totalGiving:C}");

Console.WriteLine(
    $"Largest Gift: {largestGift:C}");

Console.WriteLine(
    $"First Gift: {giftAmounts[0]:C}");

Console.WriteLine(
    $"Last Gift: {giftAmounts[^1]:C}");
```

Data flow:

EXAMPLE / FLOW

```
Gift Amount Array

[100,250,500,1000,7500]
        →
CalculateSummary
        →
Count
5

Total
9350

Largest
7500
```

Range flow:

EXAMPLE / FLOW

```
Original

100 250 500 1000 7500
 ↓   ↓   ↓

[..3]

100 250 500
```

And:

EXAMPLE / FLOW

```
Original

100 250 500 1000 7500
                ↓    ↓

[^2..]

1000 7500
```

Complete Concept Flow

EXAMPLE / FLOW

```
Need to store multiple values
        →
Are all elements same type?
        →
Yes
        →
Is size naturally fixed?
        →
Array
```

Array:

EXAMPLE / FLOW

```
Create
    →
Fixed Length
    →
Zero-Based Indexes
    →
Read / Modify Elements
    →
Iterate
    →
Search / Sort / Copy
```

Dimensions:

EXAMPLE / FLOW

```
Single sequence?
    →
T[]

Rectangular grid?
    →
T[,]

Different-length rows?
    →
T[][]
```

Modern indexing:

EXAMPLE / FLOW

```
Need last item?
    →
^1

Need subsection?
    →
Range ..
    →
Array slice creates new shallow-copy array
```

Grouped return values:

EXAMPLE / FLOW

```
Need 2–3 related values
for local/internal result?
```

```
    →
Tuple
    →
Named Elements
    →
Deconstruction
```

Key/value relationship:

EXAMPLE / FLOW

```
Key
    +
Value
    →
KeyValuePair<TKey,TValue>
```

Temporary projection:

EXAMPLE / FLOW

```
Existing Object
    →
Need only selected fields temporarily
    →
Anonymous Type
```

Important business concept:

EXAMPLE / FLOW

```
Long-lived data

API contract

Service contract

Domain object
        →
Named Class / Record / Struct
        →
Not large anonymous tuple
```

Final End-to-End Fundraising Flow

EXAMPLE / FLOW

```
Gift IDs Arrive

[1001,1002,1003,1004]
        →
Array
        →
Index / Iterate
        →
Process Gift Batch
```

Gift amounts:

EXAMPLE / FLOW

```
[100,250,500,1000]
        →
Calculate
        →
```

```
Tuple Result
(
    Count,
    Total,
    Average
)
```


Campaign totals:

```
EXAMPLE / FLOW
CampaignId
    →
Total Raised
    →
Key / Value relationship
```

Reporting query:

```
EXAMPLE / FLOW
Gift Entity
    →
Select only

GiftId
Amount
DonorName
    →
Anonymous Projection
```

The core lesson is:

> **Arrays provide fixed-size indexed storage for values of one element type. Tuples group a small number of related values. `KeyValuePair<TKey,TValue>` expresses a key/value relationship, and anonymous types are useful for temporary local projections. Choosing the correct container depends on whether you need fixed sequential storage, temporary grouping, key/value semantics, or a real named business model.**

With Topic 10 complete, you now have the essential C# foundation required before moving into object-oriented programming.

Next roadmap topic: Topic 11 — Classes, Objects, and Encapsulation.